

LAB-14.4

Gujja Pranitha

2303A52171

Batch 41

Task-1: Responsive Homepage Layout for a Startup Website

Prompt:

Generate a modern responsive startup homepage using HTML and CSS. The page should include a sticky header with a logo and navigation links, a hero section with a heading, description, and call-to-action button, a services section displaying service cards using CSS Grid, and a footer with copyright information. Use Flexbox/Grid for layout and media queries to ensure the page adapts smoothly for desktop, tablet, and mobile screens. Add simple hover effects and keep the code clean and well-structured.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>InnovateX Startup</title>
<style>
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Segoe UI, sans-serif;
}
/* Header */
header{
display:flex;
justify-content:space-between;
align-items:center;
padding:15px 40px;
background:#1f2937;
color:white;
position:sticky;
```

```
top:0;
}
.logo{
font-size:24px;
font-weight:bold;
}
nav ul{
display:flex;
list-style:none;
}
nav ul li{
margin-left:25px;
}
nav ul li a{
text-decoration:none;
color:white;
transition:0.3s;
}
nav ul li a:hover{
color:#38bdf8;
}
/* Hero Section */
.hero{
display:flex;
flex-direction:column;
justify-content:center;
align-items:center;
text-align:center;
height:70vh;
background:linear-gradient(120deg,#38bdf8,#1ab46c);
color:white;
padding:20px;
}
.hero h1{
font-size:42px;
margin-bottom:15px;
}
.hero p{
max-width:600px;
margin-bottom:20px;
}
.hero button{
padding:12px 25px;
border:none;
```

```
background:white;
color:#333;
font-weight:bold;
border-radius:25px;
cursor:pointer;
transition:0.3s;
}
.hero button:hover{
background:#1f2937;
color:white;
}
/* Services Section */
.services{
display:grid;
grid-template-columns:repeat(3,1fr);
gap:20px;
padding:60px 40px;
background:#f4f4f4;
text-align:center;
}
.service-box{
background:white;
padding:25px;
border-radius:10px;
box-shadow:0 4px 10px rgba(0,0,0,0.1);
transition:0.3s;
}
.service-box:hover{
transform:translateY(-5px);
}
.service-box h3{
margin-bottom:10px;
}
/* Footer */
footer{
text-align:center;
background:#1f2937;
color:white;
padding:20px;
}
/* Tablet */
@media (max-width:900px){
.services{
grid-template-columns:repeat(2,1fr);
```

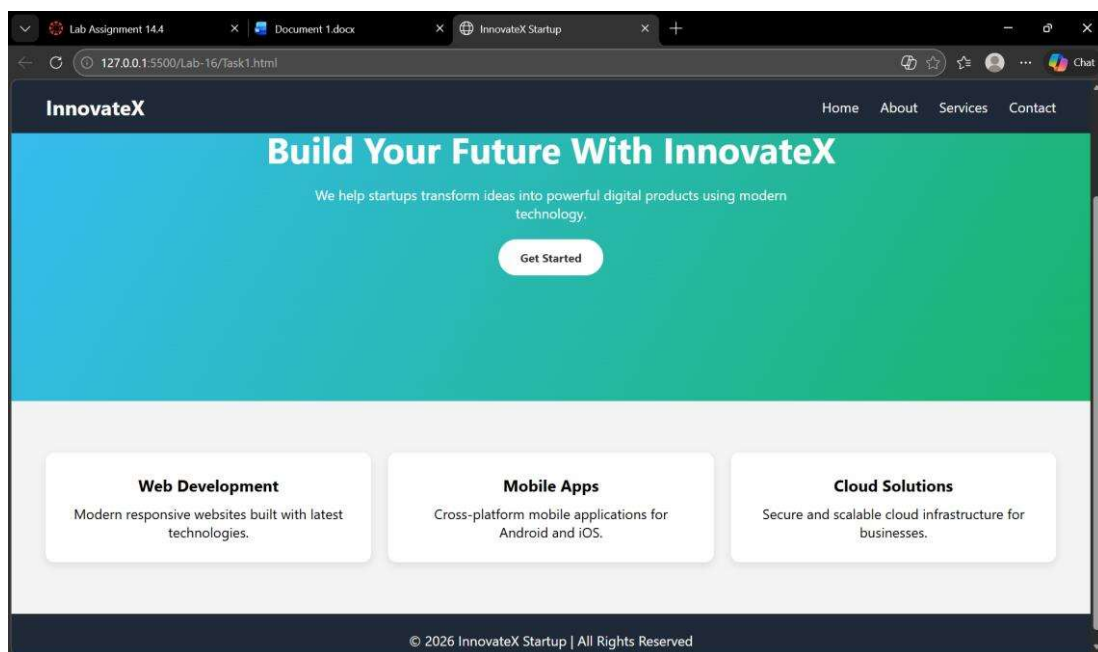
```
}
.hero h1{
font-size:34px;
}
}
/* Mobile */
@media (max-width:600px){
header{
flex-direction:column;
}
nav ul{
flex-direction:column;
margin-top:10px;
}
nav ul li{
margin:8px 0;
}
.services{
grid-template-columns:1fr;
}
.hero h1{
font-size:28px;
}
}
</style>
</head>
<body>
<header>
<div class="logo">InnovateX</div>
<nav>
<ul>
<li><a href="#">Home</a></li>
<li><a href="#">About</a></li>
<li><a href="#">Services</a></li>
<li><a href="#">Contact</a></li>
</ul>
</nav>
</header>
<section class="hero">
<h1>Build Your Future With InnovateX</h1>
<p>We help startups transform ideas into powerful digital products using modern
technology.</p>
<button>Get Started</button>
</section>
```

```

<section class="services">
<div class="service-box">
<h3>Web Development</h3>
<p>Modern responsive websites built with latest technologies.</p>
</div>
<div class="service-box">
<h3>Mobile Apps</h3>
<p>Cross-platform mobile applications for Android and iOS.</p>
</div>
<div class="service-box">
<h3>Cloud Solutions</h3>
<p>Secure and scalable cloud infrastructure for businesses.</p>
</div>
</section>
<footer>
<p>© 2026 InnovateX Startup | All Rights Reserved</p>
</footer>
</body>
</html>

```

OUTPUT:



EXPLANATION: This code creates a responsive startup landing webpage called “InnovateX” using HTML and CSS. The HTML structure includes a header with a logo and navigation menu (Home, About, Services, Contact), a hero section with a heading, description, and “Get Started” button, a services section displaying three service boxes

(Web Development, Mobile Apps, and Cloud Solutions), and a footer with copyright information. The CSS styling is used to design the layout, colors, spacing, and fonts, using Flexbox for the header alignment and CSS Grid for arranging the service boxes. It also includes hover effects for links and service cards to improve user interaction. Additionally, media queries are used to make the website responsive so that it adjusts properly on tablets and mobile devices by changing the layout from three columns to two or one column depending on the screen size.

Task-2: Interactive Call-to-Action Button

Prompt:

Generate a simple interactive marketing web page using HTML, CSS, and JavaScript. The page should contain a call-to-action button such as "Start Now" or "Join Us". When the user clicks the button, JavaScript should display a visible message on the page or an alert without reloading the page. Add meaningful comments in the JavaScript code explaining each step of the logic. Also include basic CSS styling and a hover animation for the button. Ensure the code is clean, well-structured, and easy to understand.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Interactive Call To Action</title>
<style>
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial, sans-serif;
}
body{
display:flex;
justify-content:center;
align-items:center;
height:100vh;
background:linear-gradient(120deg,#4facfe,#00f2fe);
text-align:center;
}
```

```

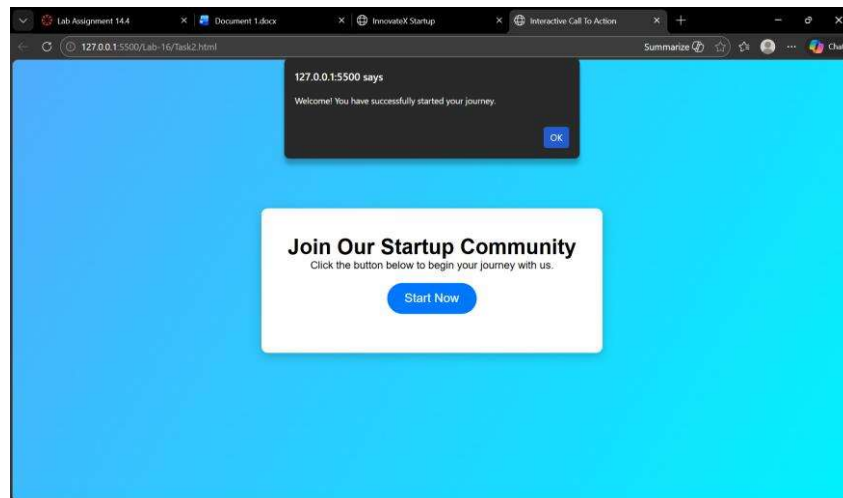
/* Container */
.container{
background:white;
padding:40px;
border-radius:10px;
box-shadow:0 5px 15px rgba(0,0,0,0.2);
}
/* CTA Button */
.cta-btn{
padding:12px 25px;
font-size:18px;
border:none;
border-radius:25px;
background:#4facfe;
color:white;
cursor:pointer;
transition:0.3s;
}
/* Hover animation */
.cta-btn:hover{
background:#0077ff;
transform:scale(1.05);
}
/* Message area */
#message{
margin-top:20px;
font-size:18px;
color:#333;
font-weight:bold;
}
</style>
</head>
<body>
<div class="container">
<h1>Join Our Startup Community</h1>
<p>Click the button below to begin your journey with us.</p>
<br>
<button class="cta-btn" id="ctaButton">Start Now</button>
<p id="message"></p>
</div>
<script>
/*
This script handles the interaction for the
Call-To-Action button.

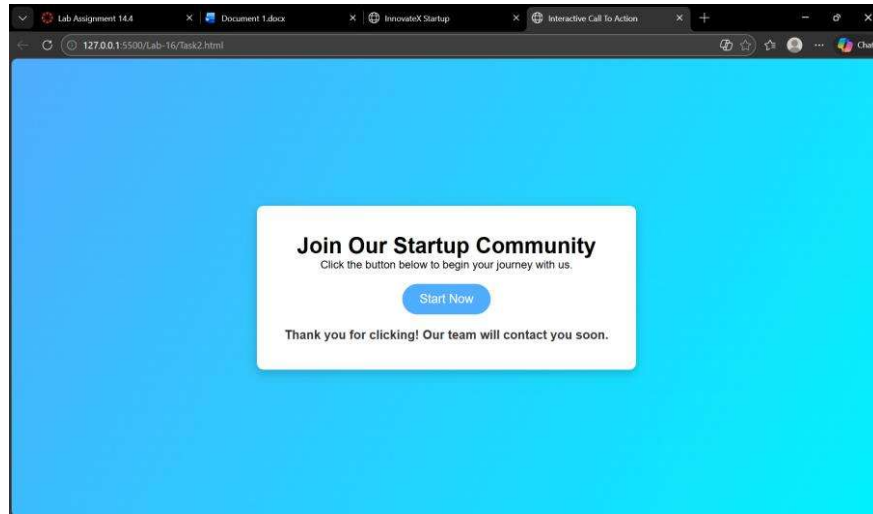
```

When the user clicks the button, a message appears without refreshing the page.

```
*/  
// Select the button element using its ID  
const button = document.getElementById("ctaButton");  
// Select the message area where text will appear  
const message = document.getElementById("message");  
// Add a click event listener to the button  
button.addEventListener("click", function(){  
    // Display an alert message to the user  
    alert("Welcome! You have successfully started your journey.");  
    // Show a visible message on the page  
    message.textContent = "Thank you for clicking! Our team will contact you  
soon.";  
});  
</script>  
</body>  
</html>
```

OUTPUT:





EXPLANATION: This code creates an **interactive Call-To-Action (CTA) webpage** using **HTML, CSS, and JavaScript**. The **HTML** structure contains a centered container with a heading, a short description, a **“Start Now” button**, and an empty message area. The **CSS** styles the page with a **gradient background**, centers the content using **Flexbox**, and designs the button with rounded corners, hover animation, and shadow effects to make it visually appealing. The **JavaScript** adds interactivity by selecting the button and message elements using `getElementById()` and attaching a **click event listener**. When the button is clicked, an **alert message appears**, and a **thank-you message is displayed on the page without refreshing**, showing how JavaScript enables dynamic user interaction.

Task-3: Contact Form with Client-Side Validation

Prompt:

Generate a contact form for a company website using HTML, CSS, and JavaScript. The form should include fields for Name, Email, and Message. Implement client-side validation using JavaScript that prevents empty submissions and checks whether the email address is in a valid format. Display clear inline error messages below each field when validation fails. If all inputs are valid, display a success message without reloading the page. Add meaningful comments in the JavaScript code and keep the structure clean and well organized.

CODE:

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Smart Contact Form</title>
<style>
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial;
}
body{
display:flex;
justify-content:center;
align-items:center;
height:100vh;
background:#f4f6f8;
}
/* Container */
.container{
width:380px;
padding:30px;
border-radius:10px;
box-shadow:0 6px 20px rgba(0,0,0,0.15);
background:white;
}
/* Floating input */
.field{
position:relative;
margin-bottom:20px;
}
input, textarea{
width:100%;
padding:10px;
border:1px solid #ccc;
border-radius:5px;
outline:none;
}
label{
position:absolute;
left:10px;
top:10px;
color:#777;
```

```

font-size:14px;
transition:0.3s;
background:white;
padding:0 4px;
}
/* Floating label animation */
input:focus + label,
input:not(:placeholder-shown) + label,
textarea:focus + label,
textarea:not(:placeholder-shown) + label{
top:-8px;
font-size:12px;
color:#0077ff;
}
/* Error message */
.error{
color:red;
font-size:12px;
margin-top:3px;
}
/* Character counter */
.counter{
font-size:12px;
text-align:right;
margin-bottom:10px;
}
/* Button */
button{
width:100%;
padding:10px;
border:none;
border-radius:20px;
background:#0077ff;
color:white;
font-size:16px;
cursor:pointer;
}
/* Shake animation */
.shake{
animation:shake 0.3s;
}
@keyframes shake{
0%{transform:translateX(0)}
25%{transform:translateX(-5px)}

```

```

50%{transform:translateX(5px)}
75%{transform:translateX(-5px)}
100%{transform:translateX(0)}
}
/* Preview box */
.preview{
margin-top:15px;
padding:10px;
border:1px dashed #aaa;
font-size:14px;
}
.success{
color:green;
font-weight:bold;
margin-top:10px;
text-align:center;
}
</style>
</head>
<body>
<div class="container">
<h2>Contact Us</h2>
<form id="contactForm">
<div class="field">
<input type="text" id="name" placeholder=" ">
<label>Name</label>
<div class="error" id="nameError"></div>
</div>
<div class="field">
<input type="text" id="email" placeholder=" ">
<label>Email</label>
<div class="error" id="emailError"></div>
</div>
<div class="field">
<textarea id="message" placeholder=" " maxlength="150"></textarea>
<label>Message</label>
</div>
<div class="counter">
<span id="count">0</span>/150
</div>
<div class="error" id="messageError"></div>
<button type="submit">Send Message</button>
<div class="success" id="successMsg"></div>
</form>

```

```

<div class="preview">
<b>Live Message Preview:</b>
<p id="previewText">Your message will appear here...</p>
</div>
</div>
<script>
/* Get elements */
const form = document.getElementById("contactForm");
const nameInput = document.getElementById("name");
const emailInput = document.getElementById("email");
const messageInput = document.getElementById("message");
const nameError = document.getElementById("nameError");
const emailError = document.getElementById("emailError");
const messageError = document.getElementById("messageError");
const preview = document.getElementById("previewText");
const counter = document.getElementById("count");
const successMsg = document.getElementById("successMsg");
/* Live message preview */
messageInput.addEventListener("input", function(){
preview.textContent = messageInput.value || "Your message will appear here...";
counter.textContent = messageInput.value.length;
});
/* Form validation */
form.addEventListener("submit", function(e){
e.preventDefault();
nameError.textContent="";
emailError.textContent="";
messageError.textContent="";
successMsg.textContent="";
let valid=true;
/* Name validation */
if(nameInput.value.trim()==""){
nameError.textContent="Name required";
valid=false;
}
/* Email validation */
let emailPattern=/^[^ ]+@[^ ]+\.[a-z]{2,3}$/;
if(emailInput.value.trim()==""){
emailError.textContent="Email required";
valid=false;
}
else if(!emailInput.value.match(emailPattern)){
emailError.textContent="Invalid email";
valid=false;
}
}

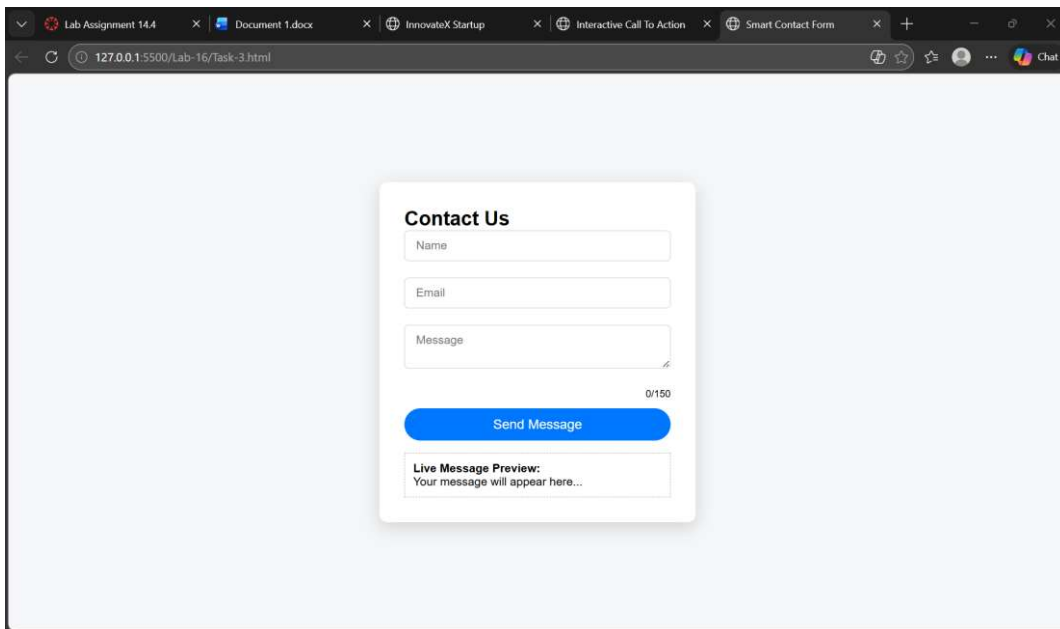
```

```

}
/* Message validation */
if(messageInput.value.trim()==""){
messageError.textContent="Message required";
valid=false;
}
/* Shake effect on error */
if(!valid){
form.classList.add("shake");
setTimeout(()=>{
form.classList.remove("shake");
},300);
return;
}
/* Success */
successMsg.textContent="Message sent successfully!";
form.reset();
preview.textContent="Your message will appear here...";
counter.textContent="0";
});
</script>
</body>
</html>

```

OUTPUT:



The screenshot displays a web browser window with multiple tabs. The active tab shows a contact form titled "Contact Us". The form includes three input fields: "Name", "Email", and "Message". The "Message" field has a character count of "0/150". Below the input fields is a blue button labeled "Send Message". At the bottom of the form, there is a section titled "Live Message Preview:" which displays the text "Your message will appear here...".

The screenshot shows a web browser window with several tabs open. The active tab is titled 'Smart Contact Form'. The address bar shows the URL '127.0.0.1:5500/Lab-16/Task-3.html'. The page content features a centered 'Contact Us' form. The form has three input fields: 'Name' (pre-filled with 'pranitha'), 'Email' (pre-filled with 'pranitha@gmail.com'), and 'Message' (pre-filled with 'I'm learning AI ASSISTED CODING'). A blue 'Send Message' button is positioned below the inputs. To the right of the message input, a character count '31/150' is displayed. Below the button, a 'Live Message Preview' section shows the text 'I'm learning AI ASSISTED CODING'.

This screenshot shows the same 'Contact Us' form after a successful submission. The 'Name' and 'Email' fields are now empty. The 'Message' field contains the text 'Message sent successfully!' in green. The character count next to the message input is now '0/150'. The 'Live Message Preview' section now displays 'Your message will appear here...'. The 'Send Message' button remains visible.

EXPLANATION: This code creates a **Smart Contact Form** using **HTML**, **CSS**, and **JavaScript**. The **HTML** builds the form structure with input fields for **Name**, **Email**, and **Message**, along with a submit button and a live preview area. The **CSS** styles the page with a centered container, floating labels for inputs, error messages, a character counter, and a **shake animation** that appears when validation fails. The **JavaScript** adds interactivity by

selecting form elements, showing a **live preview of the message while typing**, counting characters up to **150**, and validating the form when submitted. If any field is empty or the email format is incorrect, an error message appears and the form shakes; if all inputs are valid, a **success message is displayed and the form resets** without refreshing the page.

Task-4: Dynamic List Management for a Product Page

Prompt:

Generate a dynamic item list page using HTML, CSS, and JavaScript. The page should display a list of attractive tech gadget items such as AI Smart Assistant, Wireless Noise-Cancelling Headphones, and Smart Fitness Watch. Provide two buttons: “Add Item” and “Remove Item”. When the Add button is clicked, a new gadget from a predefined list should be added dynamically to the list. When the Remove button is clicked, the last item should be removed. Display the total number of items and show a message when the list becomes empty. Ensure the DOM updates instantly without page reload and include clear comments in the JavaScript code.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Product Listing</title>
<style>
*{
margin:0;
padding:0;
box-sizing:border-box;
font-family:Arial, sans-serif;
}
body{
display:flex;
justify-content:center;
align-items:center;
height:100vh;
background:linear-gradient(120deg,#6a7dfc,#9a6dfc);
}
```



```
/* Main Card */
.container{
background:white;
width:420px;
padding:25px;
border-radius:10px;
box-shadow:0 8px 20px rgba(0,0,0,0.2);
}
h2{
text-align:center;
margin-bottom:5px;
}
.sub{
text-align:center;
font-size:14px;
color:#666;
margin-bottom:15px;
}
/* Input Area */
input{
width:100%;
padding:10px;
border:1px solid #ccc;
border-radius:5px;
margin-bottom:10px;
}
/* Buttons */
.buttons{
display:flex;
gap:10px;
margin-bottom:15px;
}
button{
flex:1;
padding:8px;
border:none;
border-radius:5px;
color:white;
cursor:pointer;
}
.add{background:#2ecc71;}
.remove{background:#e74c3c;}
.clear{background:#6c757d;}
/* Product List */
```

```

.product{
background:#f1f3f5;
padding:10px;
margin-top:8px;
border-radius:5px;
display:flex;
justify-content:space-between;
align-items:center;
}
.delete{
background:#e74c3c;
border:none;
color:white;
padding:4px 8px;
border-radius:4px;
cursor:pointer;
}
.message{
margin-bottom:10px;
font-size:13px;
color:green;
}
</style>
</head>
<body>
<div class="container">
<h2>Product Listing</h2>
<p class="sub">Manage your products dynamically</p>
<div class="message" id="msg"></div>
<input type="text" id="productInput" placeholder="Enter product name">
<div class="buttons">
<button class="add" onclick="addProduct()">+ Add Product</button>
<button class="remove" onclick="removeLast()">- Remove Last</button>
<button class="clear" onclick="clearAll()">Clear All</button>
</div>
<h4>Current Products</h4>
<div id="productList"></div>
</div>
<script>
const input = document.getElementById("productInput");
const list = document.getElementById("productList");
const msg = document.getElementById("msg");
/* Add Product */
function addProduct(){

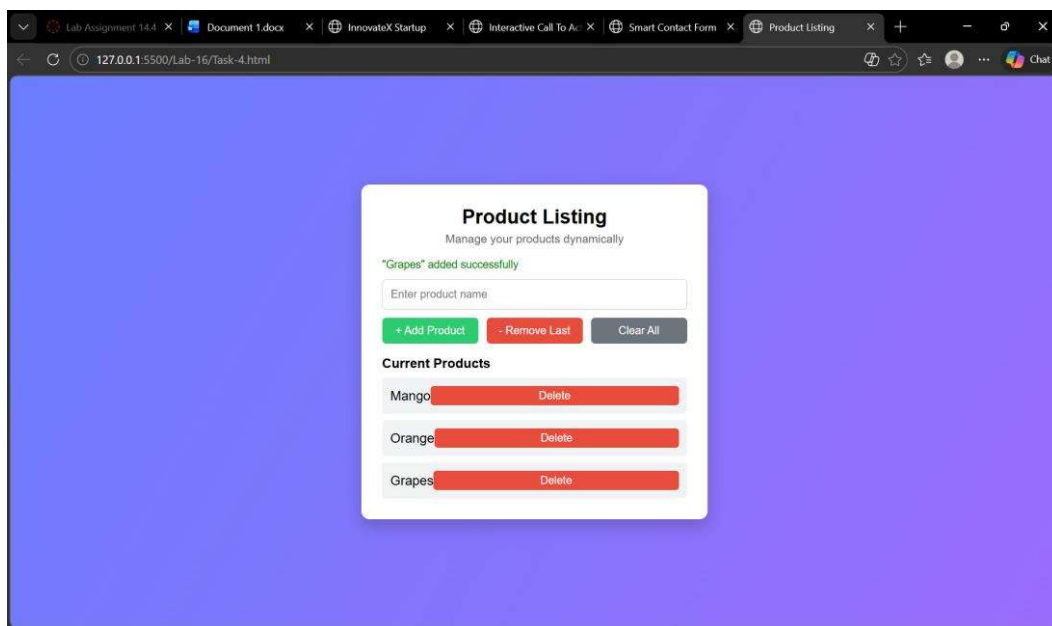
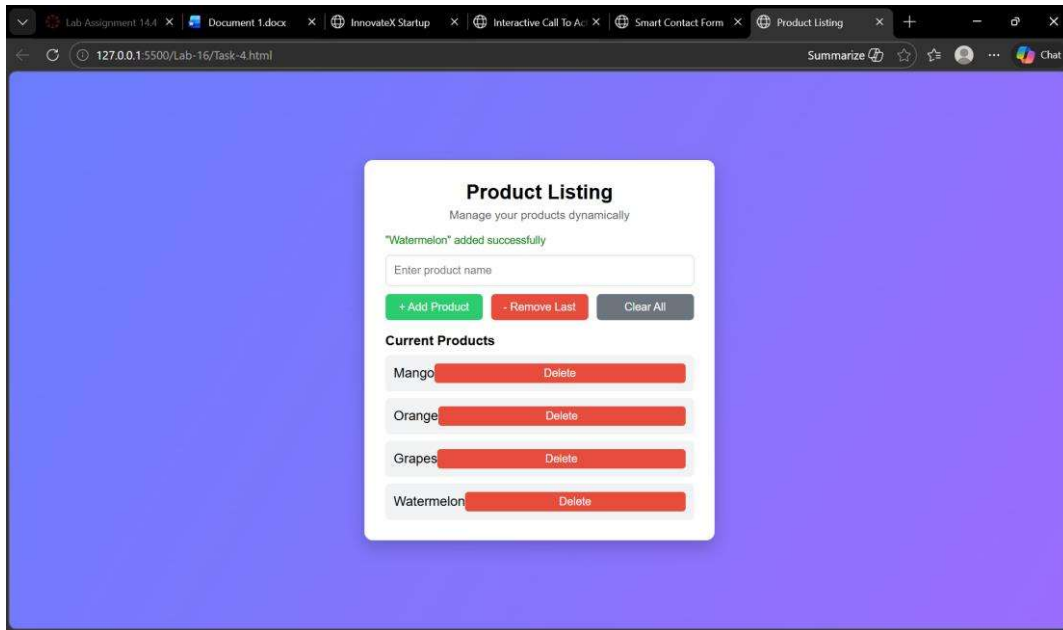
```

```

let name = input.value.trim();
if(name===""){
alert("Enter product name");
return;
}
let div = document.createElement("div");
div.className="product";
div.innerHTML = `
<span>${name}</span>
<button class="delete" onclick="deleteProduct(this)">Delete</button>
`;
list.appendChild(div);
msg.textContent = ` "${name}" added successfully`;
input.value="";
}
/* Delete specific product */
function deleteProduct(btn){
let product = btn.parentElement;
let name = product.querySelector("span").textContent;
product.remove();
msg.textContent = ` "${name}" has been deleted!`;
}
/* Remove last product */
function removeLast(){
let items = list.children;
if(items.length>0){
list.removeChild(items[items.length-1]);
msg.textContent="Last product removed";
}
}
/* Clear all products */
function clearAll(){
list.innerHTML="";
msg.textContent="All products cleared";
}
</script>
</body>
</html>

```

OUTPUT:



EXPLANATION: This code creates a **dynamic Product Listing** webpage using **HTML, CSS, and JavaScript**. The **HTML** defines the structure of the page with a container that includes a heading, an input box to enter a product name, buttons to **Add Product**, **Remove Last**, and **Clear All**, and a section to display the current products. The **CSS** styles the page with a gradient background, a centered card layout, styled buttons, and product boxes to make the interface clean and user-friendly. The **JavaScript** adds functionality by selecting elements using `getElementById()` and defining functions to **add a product to**

the list, delete a specific product, remove the last product, and clear all products. It uses DOM manipulation methods like `createElement()`, `appendChild()`, `remove()`, and `innerHTML` to update the product list dynamically without refreshing the page, while also displaying status messages for each action.

Task-5: Modal Popup for User Notifications

Prompt:

Generate a web page that includes a modal popup for user notifications. The modal should appear when a user clicks an "Open Modal" button. It must have a semi-transparent background overlay and a centered popup message. Include CSS styling for the modal and overlay. Add JavaScript functionality to open and close the modal. The modal should close when the user clicks the close button or clicks outside the popup on the overlay.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Modal Popup Notifications</title>
<style>
body{
font-family:Arial;
background:linear-gradient(120deg,#4b5bdc,#7d5bdc);
display:flex;
justify-content:center;
align-items:center;
height:100vh;
}
.container{
background:white;
padding:30px;
border-radius:10px;
width:420px;
box-shadow:0 10px 25px rgba(0,0,0,0.3);
text-align:center;
}
```

```
h2{
margin-bottom:10px;
}
.info{
background:#e6f4ff;
padding:10px;
border-left:4px solid #2196f3;
margin-bottom:20px;
text-align:left;
font-size:14px;
}
/* buttons */
.btn{
padding:10px 15px;
border:none;
border-radius:5px;
color:white;
cursor:pointer;
margin:5px;
}
.success{background:#28a745;}
.infoBtn{background:#17a2b8;}
.warning{background:#ffc107;color:black;}
.custom{background:#0f766e;}
.modal{
display:none;
position:fixed;
top:0;
left:0;
width:100%;
height:100%;
background:rgba(0,0,0,0.5);
justify-content:center;
align-items:center;
}
.modal-box{
background:white;
padding:20px;
border-radius:8px;
width:320px;
text-align:left;
position:relative;
}
.modal-header{
```

```

font-weight:bold;
margin-bottom:10px;
}
.close{
position:absolute;
right:10px;
top:10px;
cursor:pointer;
font-size:18px;
}
.modal-buttons{
text-align:right;
margin-top:15px;
}
.modal-buttons button{
padding:6px 12px;
margin-left:5px;
border:none;
border-radius:4px;
cursor:pointer;
}
.confirm{background:#4b6ef5;color:white;}
.cancel{background:#999;color:white;}
</style>
</head>
<body>
<div class="container">
<h2>Modal Popup Notifications</h2>
<div class="info">
<b>How to Close the Modal:</b>
<ul>
<li>Click the X button</li>
<li>Click the Close button</li>
<li>Click outside the modal</li>
</ul>
</div>
<h3>Notifications</h3>
<button class="btn success" onclick="openModal('successModal')">✓
Success</button>
<button class="btn infoBtn" onclick="openModal('infoModal')">i Info</button>
<button class="btn warning" onclick="openModal('warnModal')">⚠ Warning</button>
<h3 style="margin-top:20px;">Custom Modal</h3>
<button class="btn custom" onclick="openModal('customModal')">Custom
Modal</button>

```

```
</div>
<!-- SUCCESS MODAL -->
<div class="modal" id="successModal">
<div class="modal-box">
<span class="close" onclick="closeModal('successModal')">&times;</span>
<div class="modal-header">✓ Success!</div>
<p>Your operation has been completed successfully.</p>
<div class="modal-buttons">
<button class="cancel" onclick="closeModal('successModal')">Close</button>
<button class="confirm" onclick="closeModal('successModal')">Confirm</button>
</div>
</div>
</div>
<!-- INFO MODAL -->
<div class="modal" id="infoModal">
<div class="modal-box">
<span class="close" onclick="closeModal('infoModal')">&times;</span>
<div class="modal-header">Information</div>
<p>This is an informational message for users.</p>
<div class="modal-buttons">
<button class="confirm" onclick="closeModal('infoModal')">OK</button>
</div>
</div>
</div>
<!-- WARNING MODAL -->
<div class="modal" id="warnModal">
<div class="modal-box">
<span class="close" onclick="closeModal('warnModal')">&times;</span>
<div class="modal-header">Warning</div>
<p>Please be careful before performing this action.</p>
<div class="modal-buttons">
<button class="cancel" onclick="closeModal('warnModal')">Cancel</button>
<button class="confirm" onclick="closeModal('warnModal')">Continue</button>
</div>
</div>
</div>
<!-- CUSTOM MODAL -->
<div class="modal" id="customModal">
<div class="modal-box">
<span class="close" onclick="closeModal('customModal')">&times;</span>
<div class="modal-header">Custom Notification</div>
<p>
Welcome to the Custom Modal!
This modal demonstrates how you can create custom notifications.

```

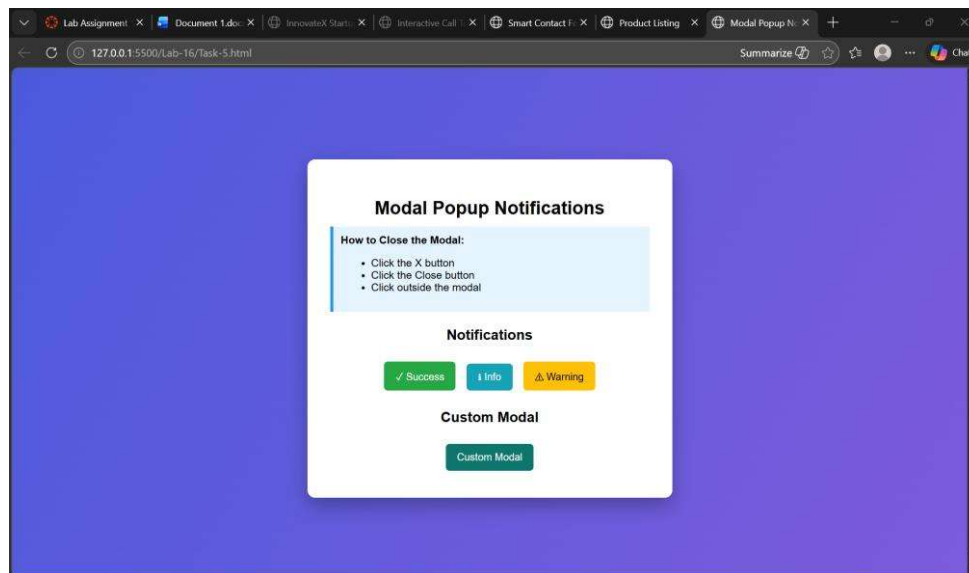


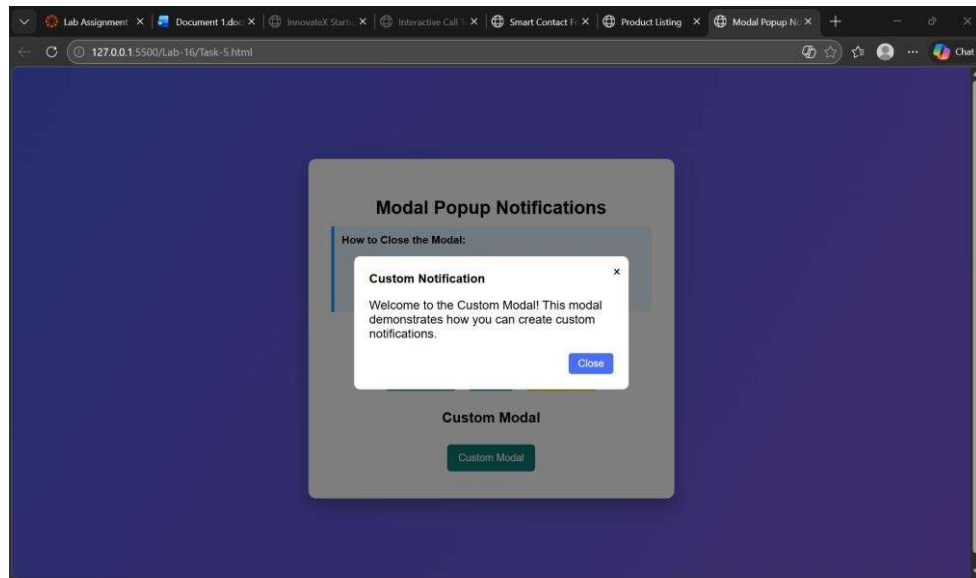
```

</p>
<div class="modal-buttons">
<button class="confirm" onclick="closeModal('customModal')">Close</button>
</div>
</div>
</div>
<script>
function openModal(id){
document.getElementById(id).style.display="flex";
}
function closeModal(id){
document.getElementById(id).style.display="none";
}
window.onclick=function(event){
let modals=document.querySelectorAll(".modal");
modals.forEach(function(modal){
if(event.target===modal){
modal.style.display="none";
}
});
};
</script>
</body>
</html>

```

OUTPUT:





EXPLANATION: This program creates **modal popup notifications using HTML, CSS, and JavaScript**. The **HTML** defines the page layout, buttons, and different modal popup structures such as **Success, Info, Warning, and Custom notifications**. The **CSS** styles the page with a gradient background, modal overlay, centered popup boxes, and styled buttons to improve the user interface. The **JavaScript** controls the modal behavior using functions to **open and close the popup windows dynamically**. Users can close the modal by clicking the **close (X) button, clicking a button inside the modal, or clicking outside the modal overlay**, providing multiple ways to dismiss the popup without refreshing the page.